



Introduction au test logiciel

Fabrice Ambert, Fabrice Bouquet
prenom.nom@femto-st.fr

Automatisation des tests logiciels

- **Résumé:**
 - appréhender les méthodes et outils pour automatiser les tests logiciels (tests unitaires, fonctionnels, d'intégration, de charge) afin de gagner en productivité et en sûreté pour le développement logiciel
- **Durée:**
 - 4 jours
- **Pré-requis:**
 - connaissances de base en test logiciel et capacité à effectuer des développements logiciels objet

Bienvenue

Faisons connaissance

- Votre formateur
- Pour chaque participant
 - Vos activités actuelles ?
 - Votre expérience ?
 - Sur des projets de développement de logiciels et systèmes d'information
 - Sur les activités de test (Rôles, Outils utilisés)
 - Vos attentes pour cette formation ?



Contenu de la formation

Programme standard

Introduction, rappels sur le processus du test

Automatisation de la gestion des tests

Automatisation des tests unitaires

Automatisation des tests d'intégration

Automatisation du test fonctionnel

Automatisation des tests système

Synthèse

Déroulement de la formation (3 jours)

Alternance de Cours et de Pratique

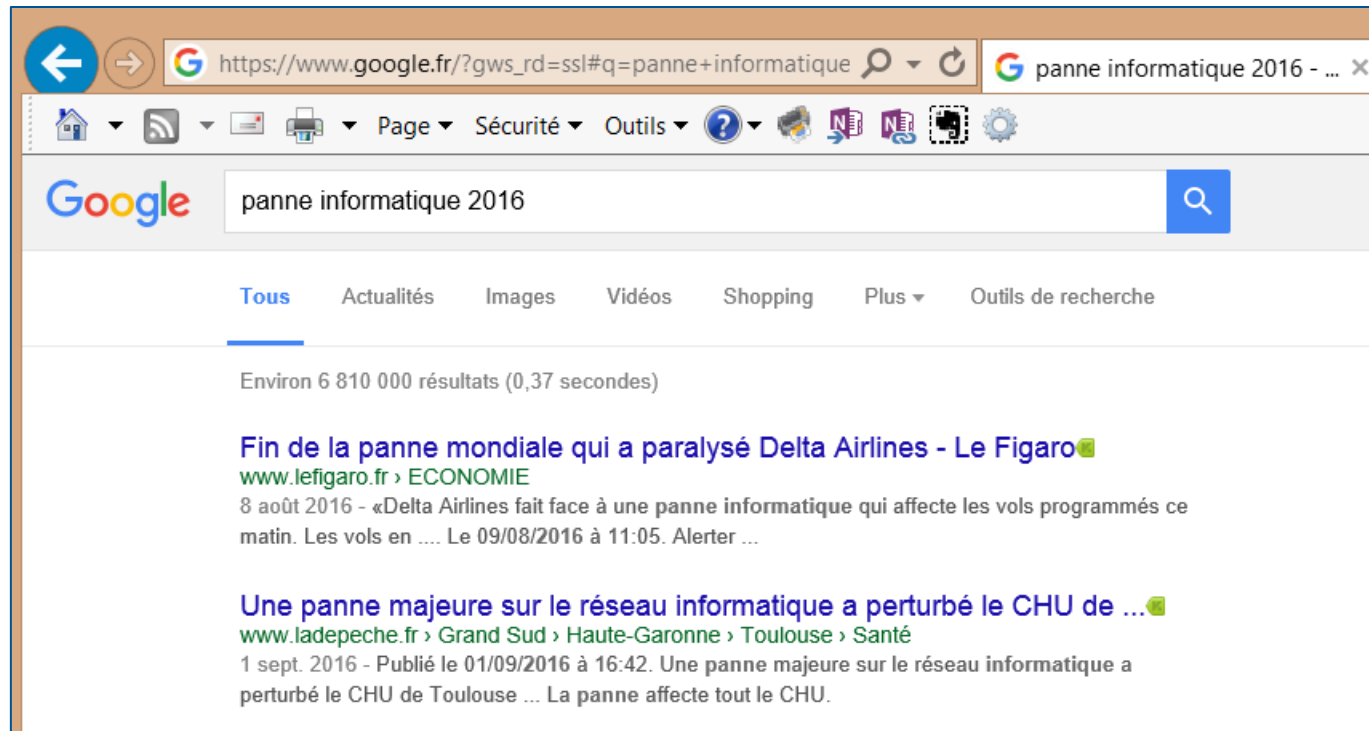
Cours	Thèmes	Durée	Exercices	Outils	Durée
1. Introduction, rappels sur le processus du test logiciel	Types, Niveaux, Méthodes	J1, 2h			
2. Automatisation des tests unitaires	Couverture de Code	J1, 1.5h	Intégrés au cours Cahier d'exercices	Junit, mockito, Jacoco	J1, 3h
3. Automatisation des tests d'intégration	Stratégie, Intégration continue	J1, 1h	Cahier d'exercices	Maven, Jenkins, SONarQube	J2, 3h
4. Test fonctionnel et son automatisation	Partitions, Limites, Combinatoire	J2, 1.5h	Intégrés au cours	FitNesse, Concondion Selenium, HTMLUnit	J2, 3h J3, 2h
5. Automatisation de la gestion des tests	Conception, exécution, suivi	J3, 2h	Cahier d'exercices	Squash TM	J3, 3.5h

1. Introduction, rappels sur le processus du test logiciel

Rôle du test dans le processus de développement.
Les tests : unitaires, fonctionnels, etc.
Les différentes méthodes de test.
Processus de test et stratégie de test.
Outils et méthodes intervenant à différentes étapes

Rôle du test dans le processus de développement

Les bugs sont nombreux et coûteux pour la société possédant le logiciel, ses fournisseurs, ses clients, les utilisateurs finaux...



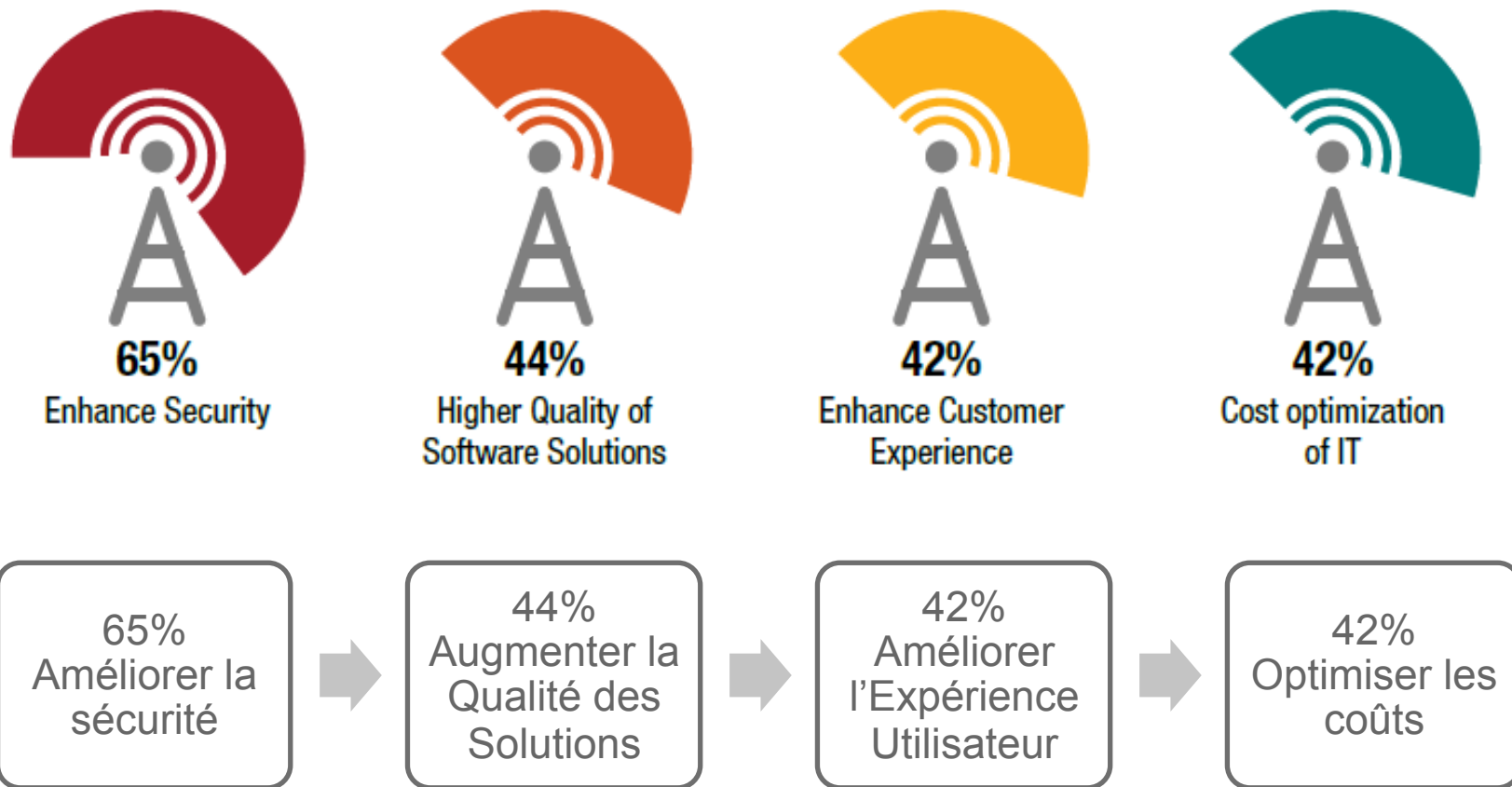
Les failles de sécurité sont également nombreuses

Problèmes de sécurité (Figaro 21/09/16)



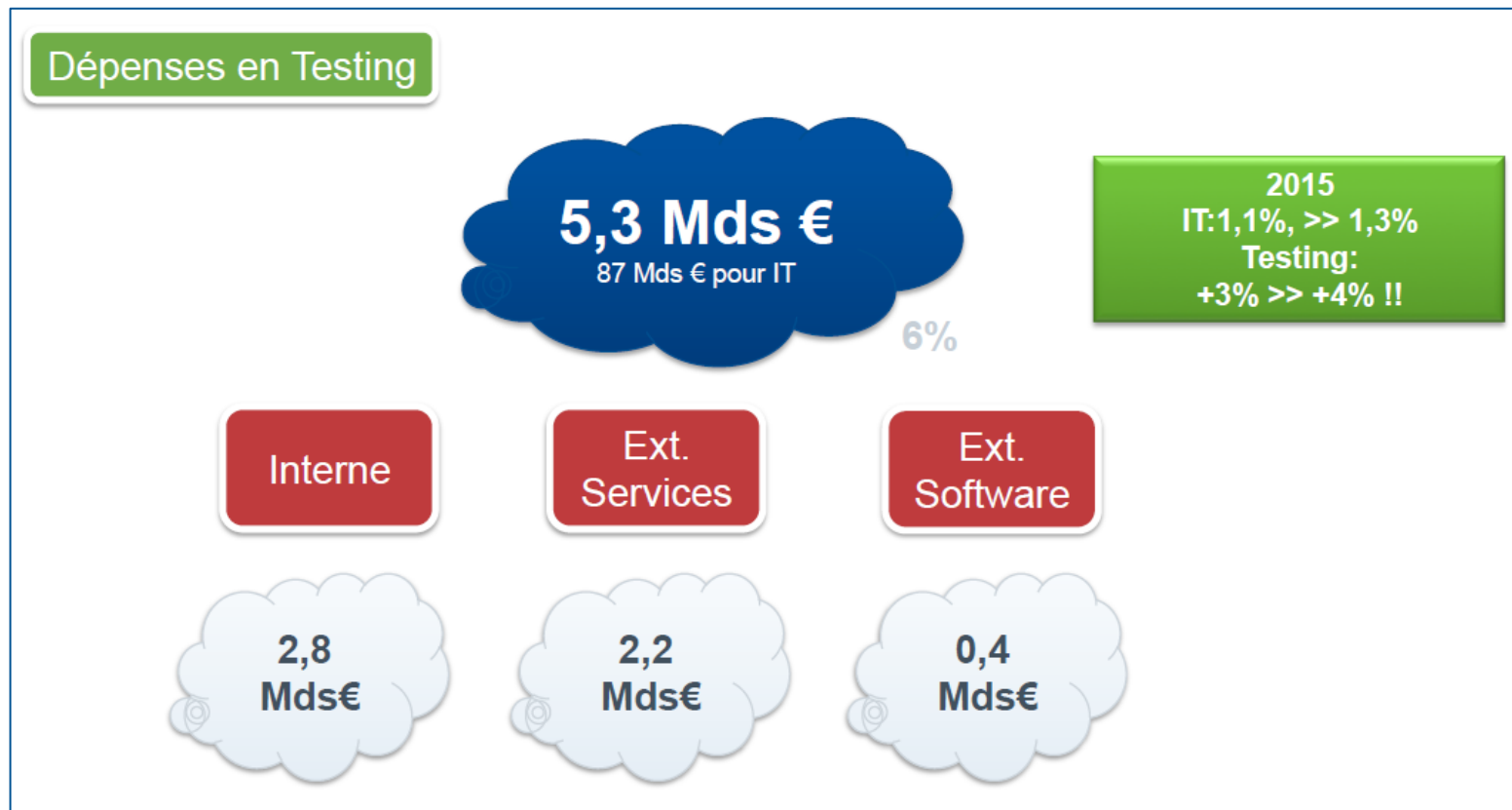
Les DSI ont des objectifs directement liés au test

Source: « World Quality Report 2016-2017, Sogeti »



Le Test continue a être un investissement nécessaire et important

Une place très importante dans l'IT



Source: PAC 2016, Arnold Aumasson

Définitions du test



- « Le test est *l'exécution ou l'évaluation d'un système ou d'un composant* par des moyens automatiques ou manuels, pour vérifier qu'il *répond à ses spécifications ou identifier les différences entre les résultats attendus et les résultats obtenus.* »

»

IEEE (Standard Glossary of Software Engineering Terminology)

- « Tester, c'est exécuter le programme *dans l'intention d'y trouver des anomalies ou des défauts.* »

G. Myers (The Art of Software testing)

- « Testing can *reveal the presence of errors but never their absence.* » Edsger W. Dijkstra (Notes on Structured Programming)

La réalité du test

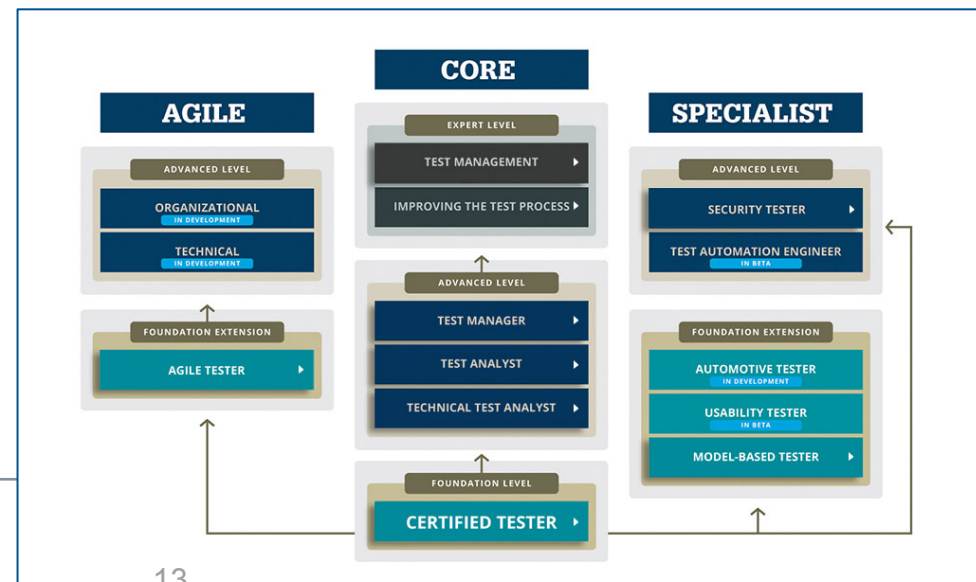


- Le test constitue aujourd'hui le vecteur principal de l'amélioration de la qualité du logiciel
- Actuellement, le test dynamique est la méthode la plus diffusée
- Il peut représenter jusqu'à 60 % de l'effort complet de développement d'un logiciel
- Coût moyen de l'activité de test :
 - 1/3 durant le développement du logiciel
 - 2/3 durant la maintenance du logiciel

Industrialiser et professionnaliser le test logiciel avec des compétences spécifiques

Un schéma mondial de Formations et Certifications en Test Logiciel

- ISTQB: International Software Testing Qualifications Board
 - www.istqb.org
 - Un parcours de formations reconnu dans le monde entier
 - Près de **500 000 certificats délivrés dans le monde**



Industrialiser et professionnaliser le test logiciel avec des compétences spécifiques

Une implantation en France très importante

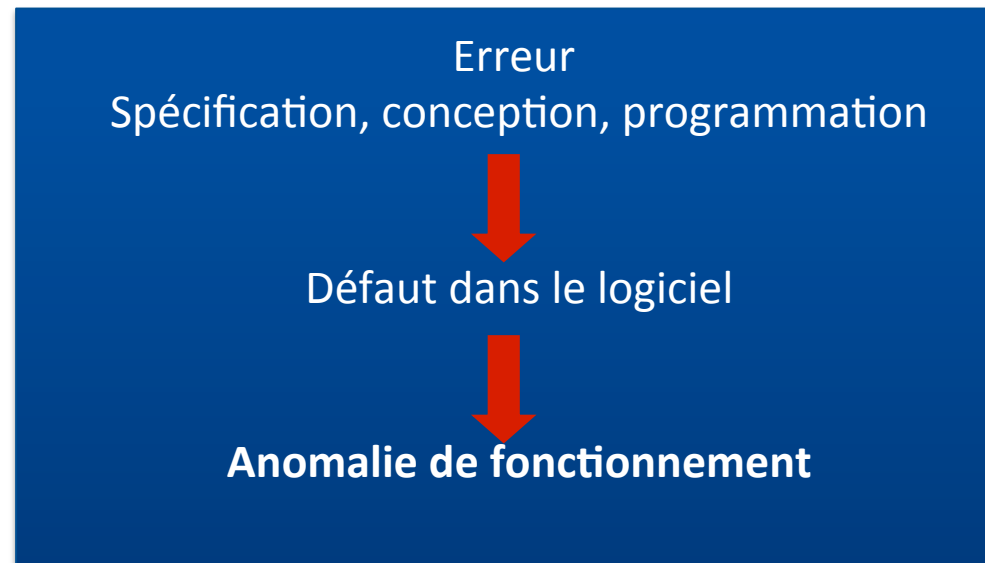


- CFTL: Comité Français des Tests Logiciels
 - www.cftl.fr
 - Déploiement en France
 - Des certifications ISTQB
 - De certifications connexes au test
 - Près de **10000 certificats délivrés en France**



Motivations du test

- Complexité croissante des architectures et des comportements
- Coût d'un bug (Ariane 5, carte à puces allemande bloquée, prime de la PAC...)



- Coût des bugs informatiques : ≈ 60 milliards \$ / an
- 22 milliards économisés si les procédures de test de logiciels étaient améliorées.

(source : NIST - National Institute of Standards and Technology)

Les tests : unitaires, fonctionnels, etc.

- V & V
 - Validation : Est-ce que le logiciel offre les services attendues ?
 - Vérification : Est-ce que les artefacts utilisés sont corrects ?
- Méthodes de V & V
 - Test statique : Revue de code, de spécifications, de documents de design
 - Test dynamique : Exécuter le code pour s'assurer d'un fonctionnement correct
 - Vérification symbolique : Run-time checking, Execution symbolique, ...
 - Vérification formelle : Preuve ou model-checking d'un modèle formel

Les tests : unitaires, fonctionnels, etc

La pratique du test

- Le test appartient à l'activité de Validation du logiciel :
est-ce-que le logiciel fait les choses bien et les bonnes choses ?
- Activité historiquement peu populaire en entreprise
- Difficultés d'ordre psychologique ou « culturel » :
 - L'activité de programmation est un processus constructif : on cherche à établir des résultats corrects
 - Le test est un processus destructif : un bon test est un test qui trouve une erreur

Les différentes méthodes de test

Test statique

Traite le code du logiciel sans l'exécuter sur des données réelles.

Test dynamique

Repose sur l'exécution effective du logiciel pour un sous ensemble bien choisi du domaine de ses entrées possibles.

Test statique



Définitions

Compilateur

- Outil logiciel qui traduit un programme exprimé dans un langage de haut niveau dans son équivalent en langage machine [IEEE 610]

Complexité

- Degré par lequel un composant ou système a une conception et/ou une structure interne difficile à comprendre, maintenir et vérifier

Test statique



L'analyse statique trouve des défauts (et non des défaillances)

- Le code n'est pas exécuté mais des outils peuvent être utilisés par des profils développeur
- La compilation peut être considérée comme du test statique outillé
- Des défauts difficiles à trouver avec des tests dynamiques peuvent être détectés
 - Dépendance dans des modèles
 - Violation de règles de sécurité et programmation
 - Maintenabilité difficile
 - Code mort
 - Variable jamais utilisée
 - Boucles infinies

Test statique



Exemples :

- *Lectures croisées / inspections*
Vérification collégiale d'un document (programme ou spécification du logiciel)
- *Analyse d'anomalies*
Corriger de manière statique les erreurs (typage impropre, code mort, ...)

Avantages :

- Méthodes efficaces et peu coûteuses
- 60% à 95% des erreurs sont détectées lors de contrôles statiques

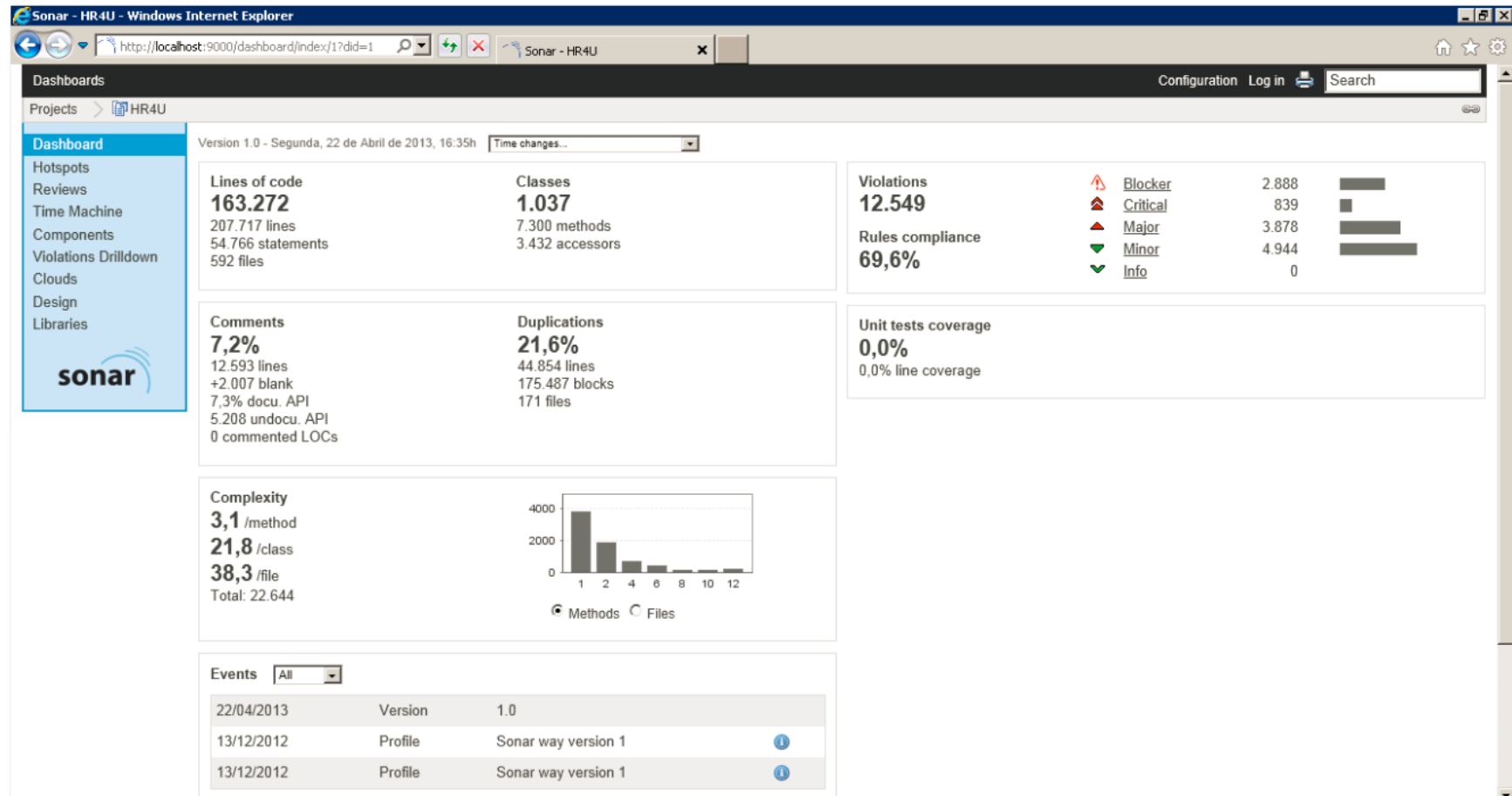
Inconvénients :

- Ne permet pas de valider le comportement d'un programme au cours de son exécution

→ Les méthodes de test statiques sont nécessaires, mais pas suffisantes

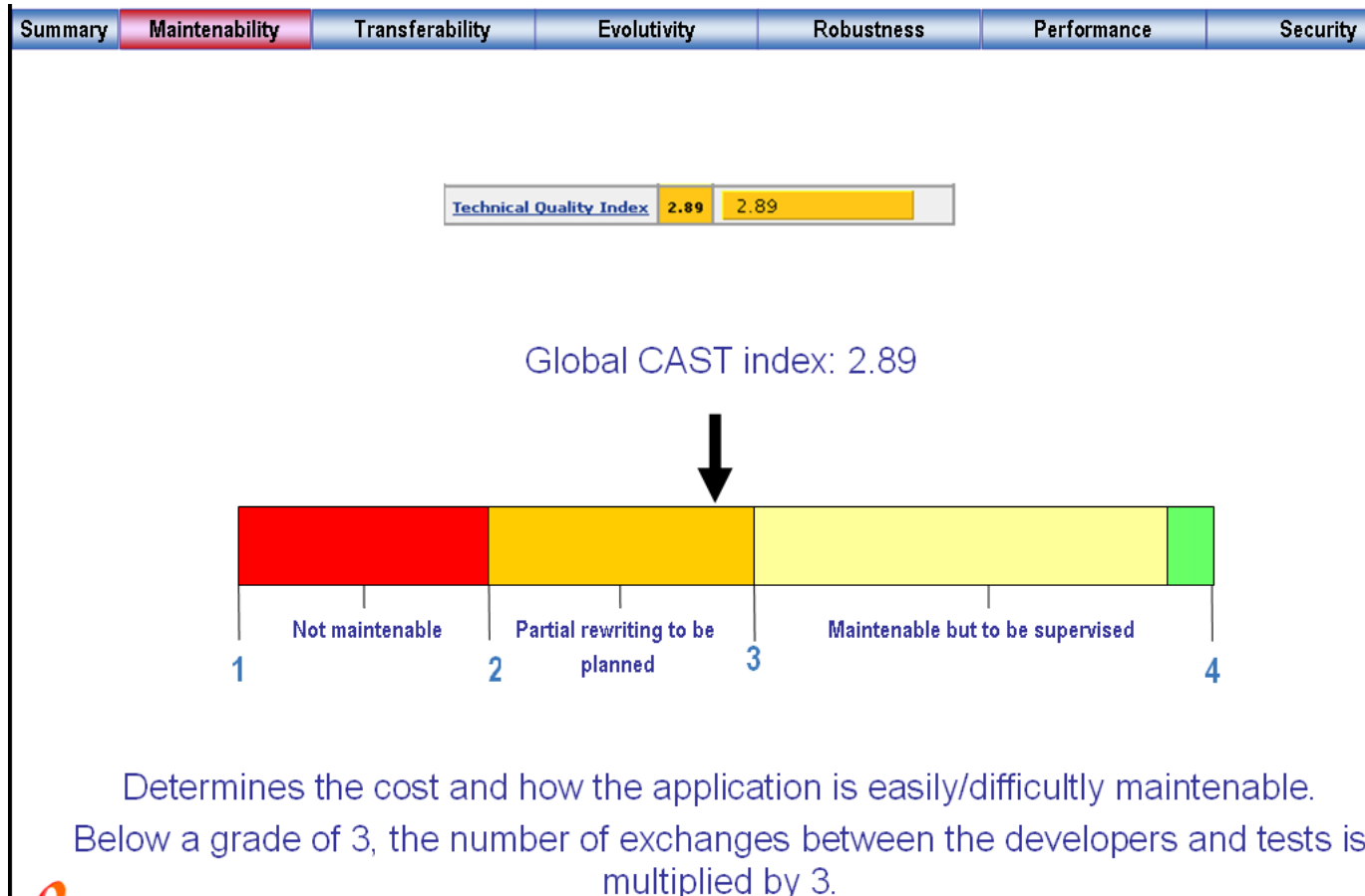
Test statique

Exemple de rapport Sonar



Test statique

Exemple d'analyse de code outillée



Test statique

Exemple d'analyse de code outillée

Summary	Maintainability	Transferability	Evolvutivity	Robustness	Performance	Security
---------	-----------------	-----------------	--------------	------------	-------------	----------

Metric grade	2.31
Name	Dead code (static)
Description	Respect of code coverage practices

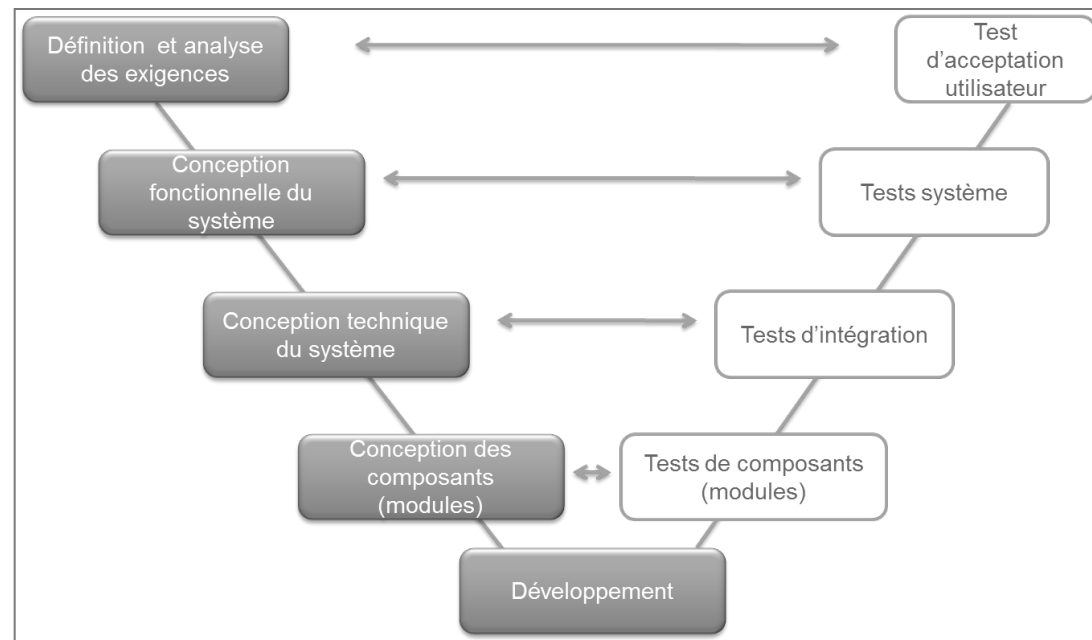
Child Metric Weight ▼	Critical contribution	Child Metric Name	Child Metric Status	Child Metric Grade
6	No	Java: Avoid unreferenced Interfaces	Moderate Risk	3.94
4	No	JavaScript: Avoid unreferenced JavaScript Functions	Very High Risk	1
4	No	Java: Avoid unreferenced Classes	Very High Risk	1.89
4	No	Java: Avoid unreferenced Methods	High Risk	2.41
4	No	Java: Avoid unreferenced Fields	High Risk	2.7
4	No	PL/SQL: Avoid unreferenced Functions / Procedures	Very High Risk	1.31
1	No	PL/SQL: Avoid unreferenced Tables	High Risk	2.38
1	No	JSP: Avoid unreferenced JSPs	Very High Risk	1.22
1	No	PL/SQL: Avoid unreferenced views	Very High Risk	1

The dead code makes it harder to understand how the application really works and makes its maintaining more expensive

Test dynamique - niveaux

Repose sur l'exécution du programme à tester

- 4 niveaux complémentaires
 - Test de composants (unitaire)
 - Test d'intégration des composants
 - Test du système global
 - Test d'acceptation (recette)



Test dynamique - techniques

Deux techniques :

- **Test structurel**
Jeu de test sélectionné en s'appuyant sur une analyse du code source du logiciel (*test boîte blanche / boîte de verre*)
- **Test fonctionnel**
Jeu de test sélectionné en s'appuyant sur les spécifications (*test boîte noire*)

En résumé, les méthodes de test dynamique consistent à :

- Exécuter le programme sur un ensemble fini de données d'entrées
- Contrôler la correction des valeurs de sortie en fonction de ce qui est attendu

Test structurel (white box)

- Les données de test sont produites à partir d'une analyse du code source

Critères de test :

- tous les chemins,
- toutes les instructions,
- etc...

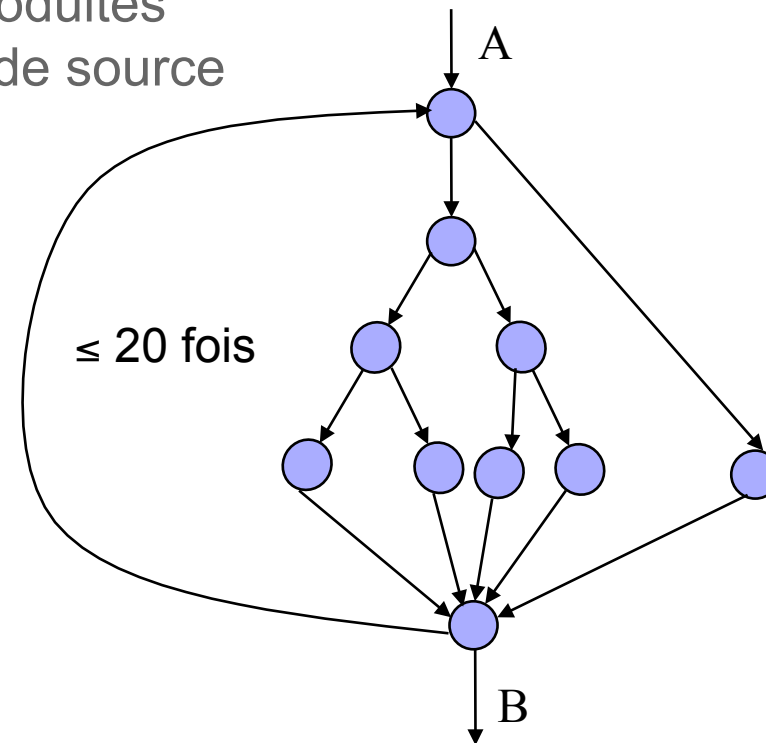
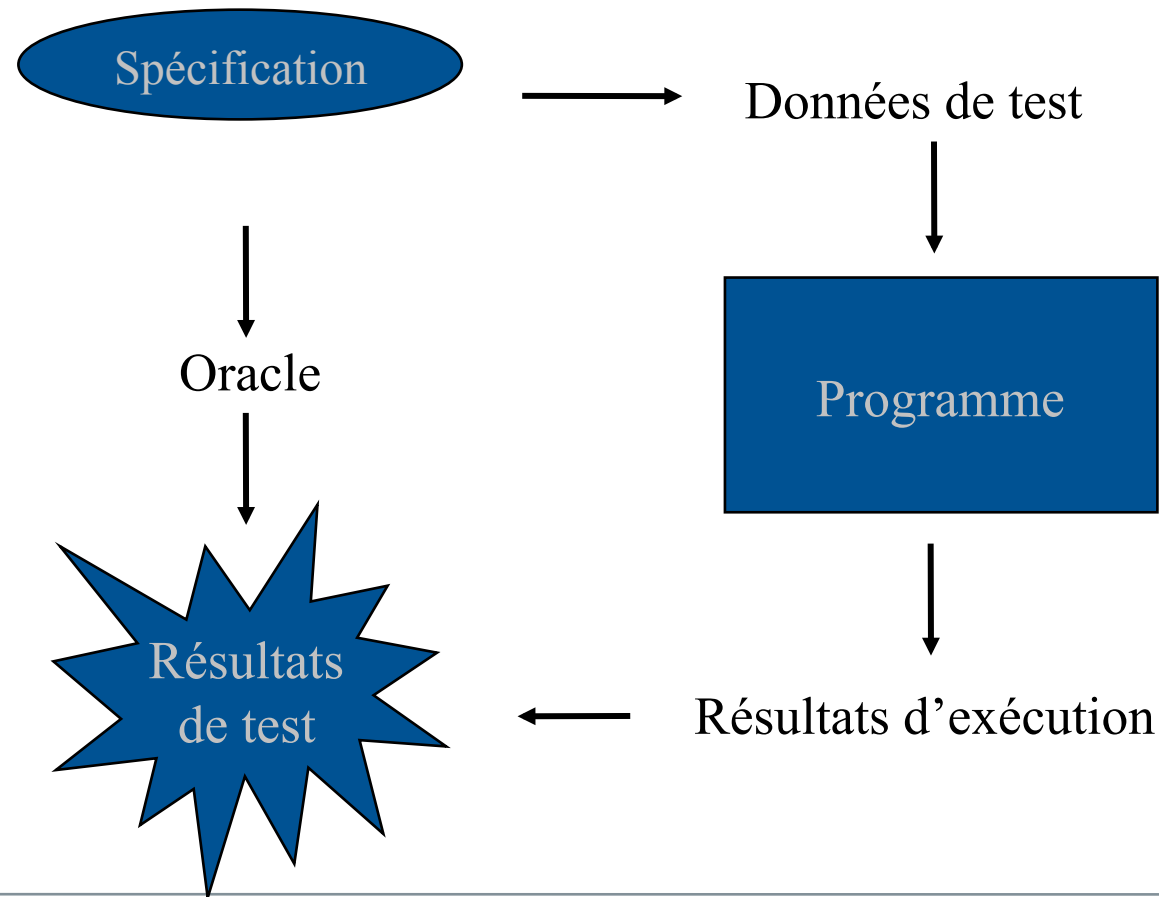


Fig. 1 : Flot de contrôle d'un petit programme

Test fonctionnel (black-box)

- Test de conformité par rapport à la spécification



Complémentarité (1) test fonctionnel / structurel

- Les 2 approches sont utilisées de façon complémentaire
- Exemple : soit le programme suivant, censé calculer la somme de 2 entiers :

```
function sum (x,y : integer) : integer;  
    if (x = 600) and (y = 500)  
    then  
        sum := x-y  
    else  
        sum := x+y;  
    end
```

- Une approche fonctionnelle détectera difficilement le défaut, alors qu'une approche par analyse de code pourra produire la donnée de test :
x = 600, y = 500.

Complémentarité (2) test fonctionnel / structurel

- En examinant ce qui a été réalisé, on ne prend pas forcément en compte ce qui aurait du être fait :
 - Les approches structurelles détectent plus facilement les erreurs commises dans le programme
 - Les approches fonctionnelles détectent plus facilement les erreurs d'omission et de spécification
- Une difficulté du test structurel consiste dans la définition de l'oracle de test.

Test de logiciels – auto-évaluation

L'exemple du triangle

- Soit la spécification suivante :

Un programme prend en entrée trois entiers. Ces trois entiers sont interprétés comme représentant les longueurs des cotés d'un triangle. Le programme rend un résultat précisant s'il s'agit d'un triangle scalène, isocèle ou équilatéral (ou une erreur si les données ne correspondent pas aux longueurs d'un triangle).

G. Myers (The Art of Software testing - 1979)

- Donnez un ensemble de cas de test que vous pensez adéquat pour tester pour ce programme...

Test de logiciels – auto-évaluation

L'exemple du triangle

- Avez-vous un cas de test pour :

Test de logiciels – auto-évaluation

L'exemple du triangle

- Chacun de ces cas correspond à un défaut constaté dans des implantations de cet exemple du triangle
- La moyenne des résultats obtenus par un ensemble de développeurs expérimentés est de 7.8 sur 14.

=> Le test est une activité complexe, a fortiori sur de grandes applications

Test dynamique – 4 activités

Sélection d'un jeu de tests : choisir un sous-ensemble des entrées possibles du logiciel

Soumission du jeu de tests

Dépouillement des résultats : consiste à décider du succès ou de l'échec du jeu de test (*verdict*):

- Fail, Pass, Inconclusive

Évaluation de la qualité et de la pertinence des tests effectués (déterminant pour la décision d'arrêt de la phase de test)

Types de tests (1) (renseignent la nature du test mené)

Tests fonctionnels

Valide les résultats rendus par les services

Tests non-fonctionnels

Valide la manière dont les services sont rendus

Tests nominaux / de bon fonctionnement

Vérifie que le résultat calculé est le résultat attendu, en entrant des données valides au programme (*test-to-pass*)

Tests de robustesse

Vérifie que le programme réagit correctement à une utilisation non conforme, en entrant des données invalides (*test-to-fail*)

Types de tests (2) (renseignent la nature du test mené)

Test de performance

- Load testing (test avec montée en charge)
- Stress testing (soumis à des demandes de ressources anormales)

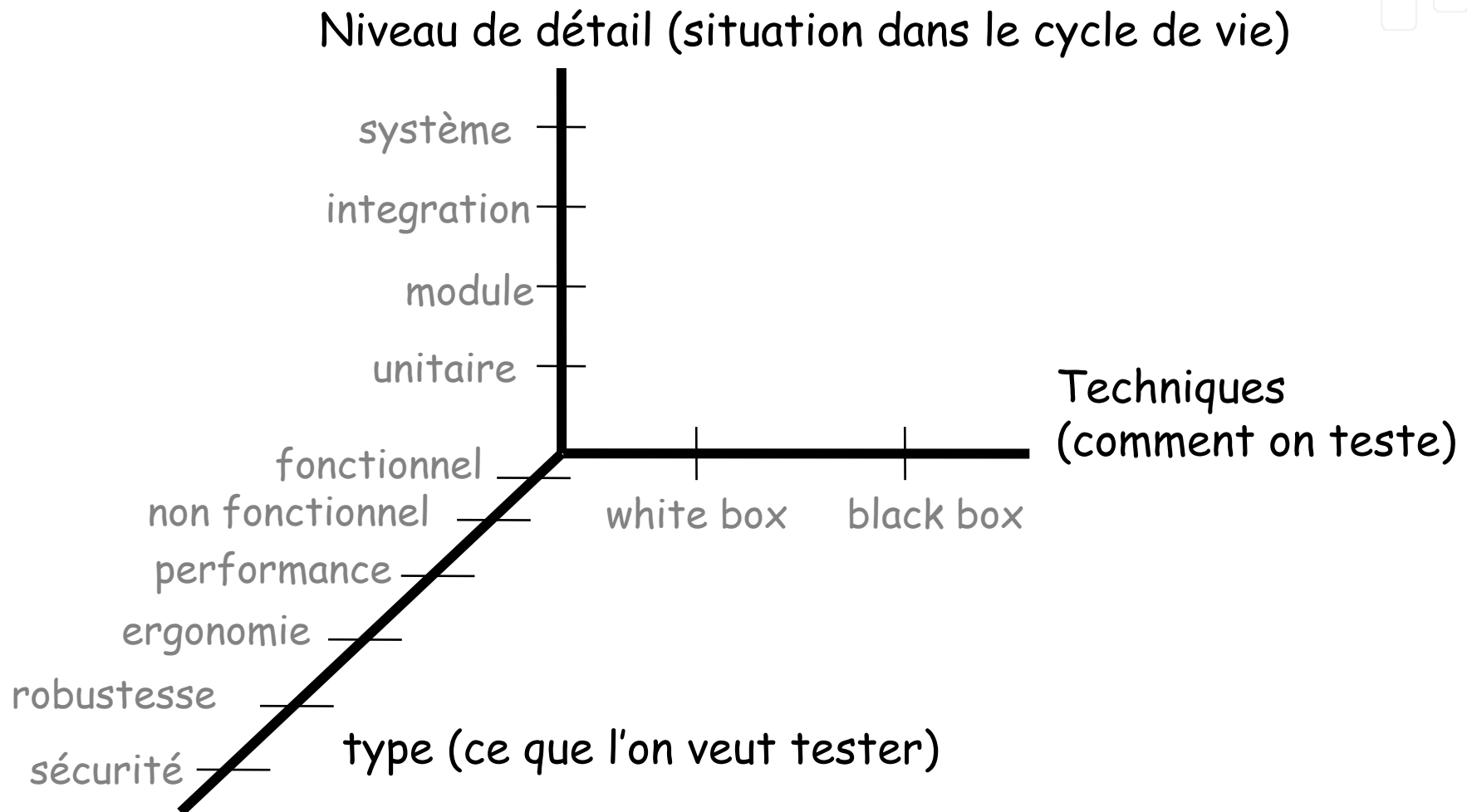
Test de non-régression

Vérifie que les corrections ou évolutions dans le code n'ont pas créé d'anomalies nouvelles

Test de confirmation

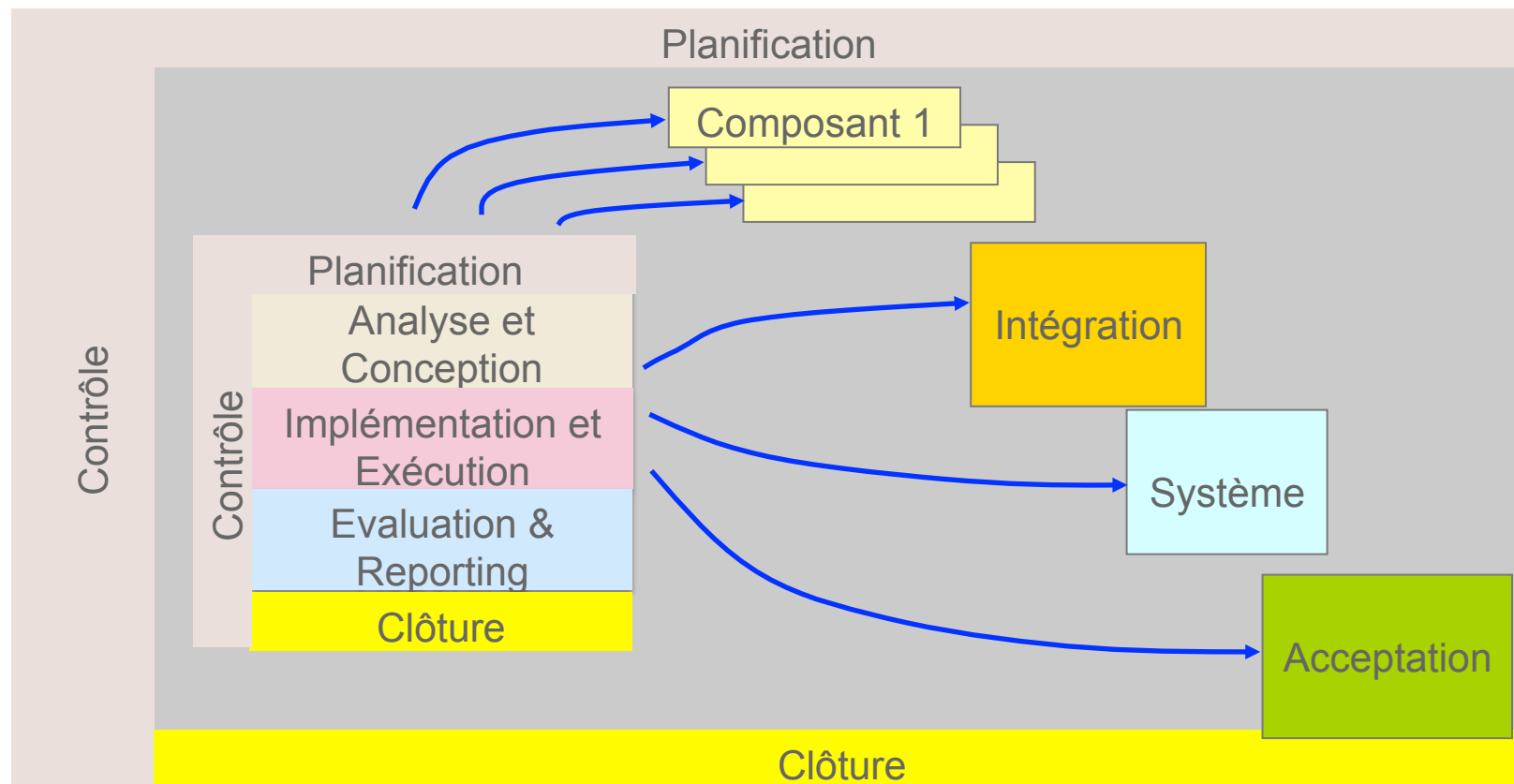
Valide la correction d'un défaut

Catégories de tests

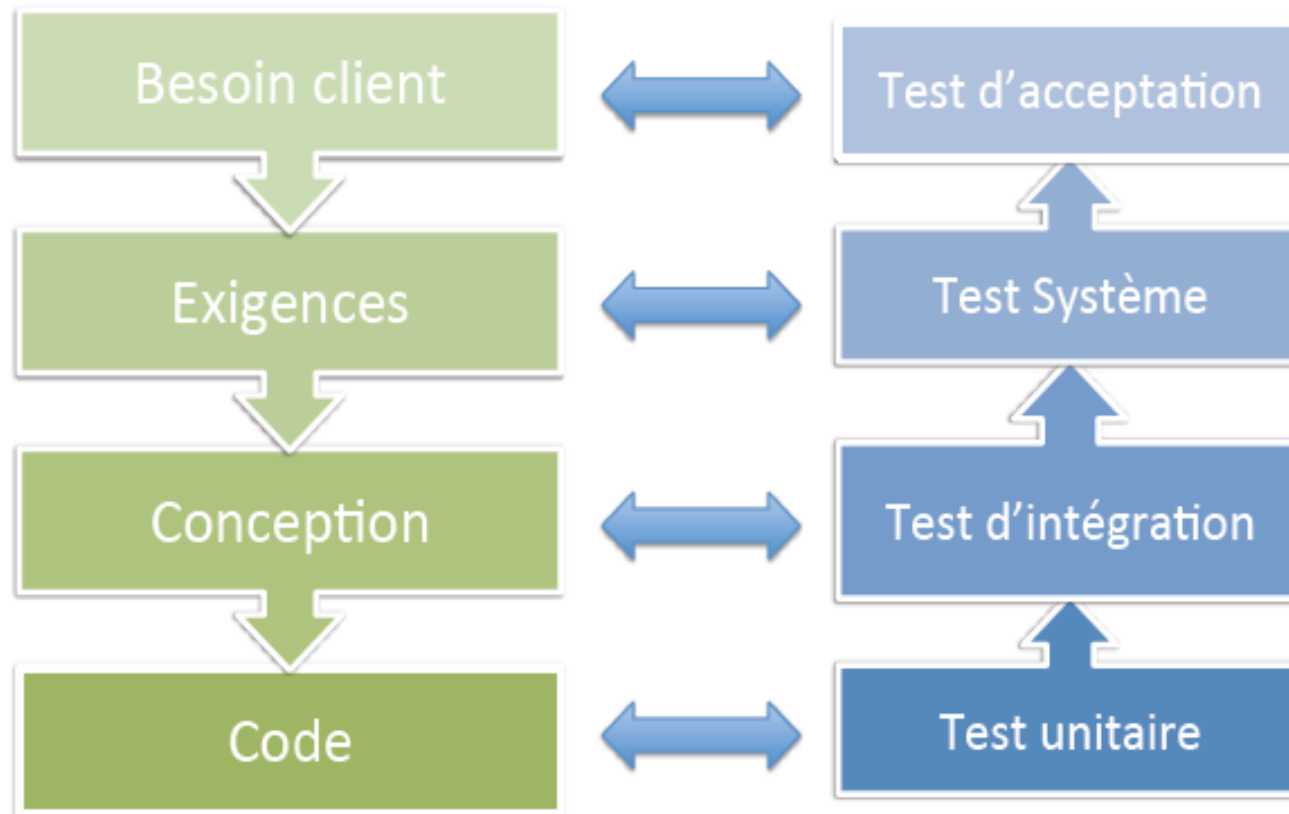


Processus de test et stratégie de test

Le Test est un ensemble d'activités à dérouler à différents niveaux



Développement et niveaux de tests



Niveaux de tests (renseignent l'objet du test)



Tests (structurels) unitaires

Test de procédures, de modules, de composants

(coût : 50% du coût total du développement initial correspondant)

Tests d'intégration

Test de bon comportement lors de la composition des procédures, modules ou composants

(coût d'un bug dans cette phase : 10 fois celui d'un bug unitaire)

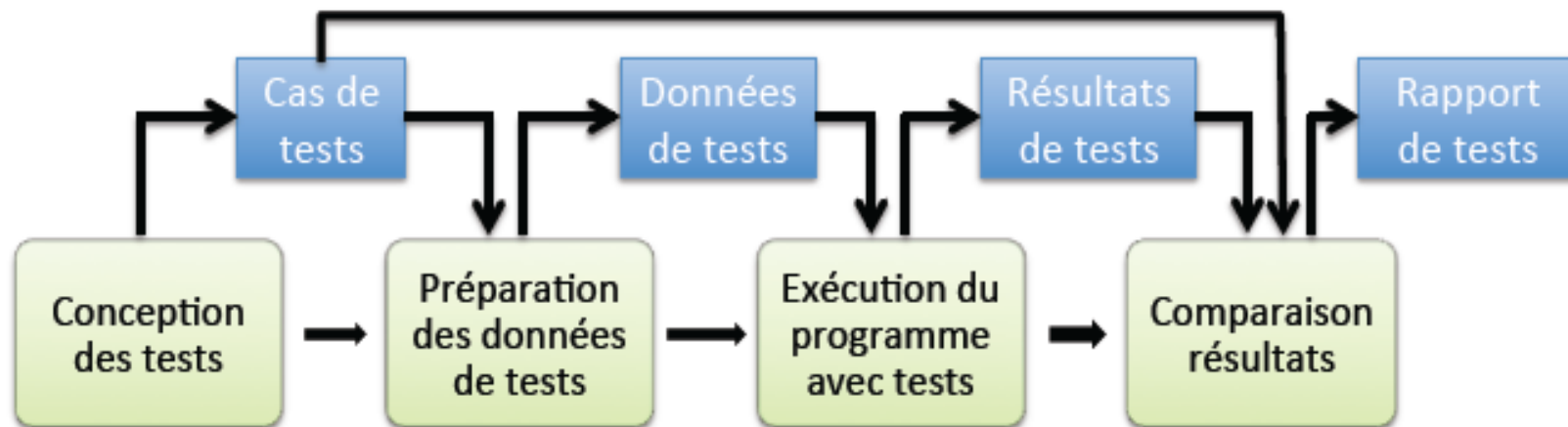
Tests système / de recette

Validation de l'adéquation aux spécifications

(coût d'un bug dans cette phase : 100 fois celui d'un bug unitaire)

Processus de test et stratégie de test

Test et cycle de vie du processus de test



Difficultés du test



- Le test exhaustif est en général impossible à réaliser
 - En test structurel, le parcours du graphe de flot de contrôle conduit à une forte explosion combinatoire

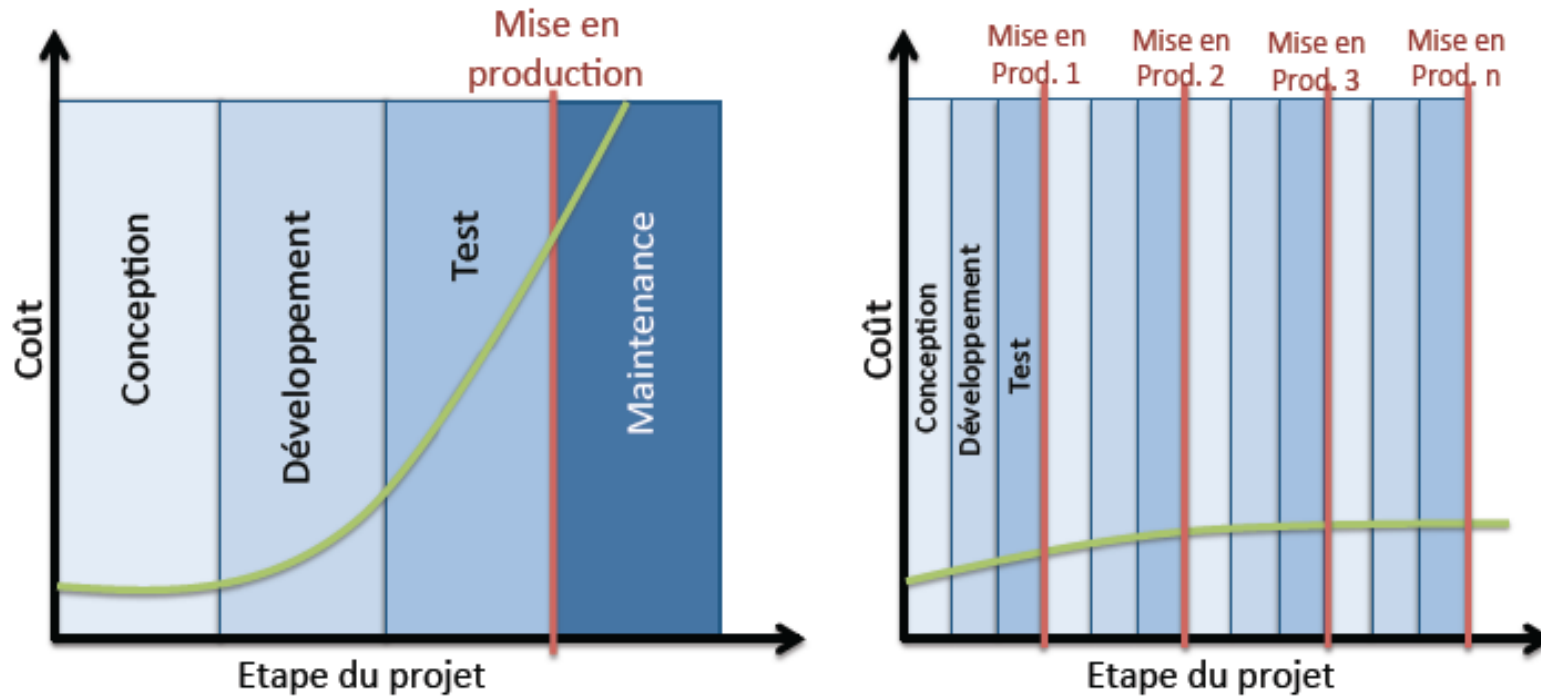
Exemple : le nombre de chemin logique dans le graphe de la figure 1 est supérieur à $10^{14} \approx 5^{20} + 5^{19} + \dots + 5^1$
 - En test fonctionnel, l'ensemble des données d'entrée est en général infini ou très grande taille

Exemple : un logiciel avec 5 entrées analogiques sur 8 bits admet 2^{40} valeurs différentes en entrée
- => le test est une méthode de validation partielle de logiciels
- => la qualité du test dépend de la pertinence du choix des données de test

Coût d'un bug

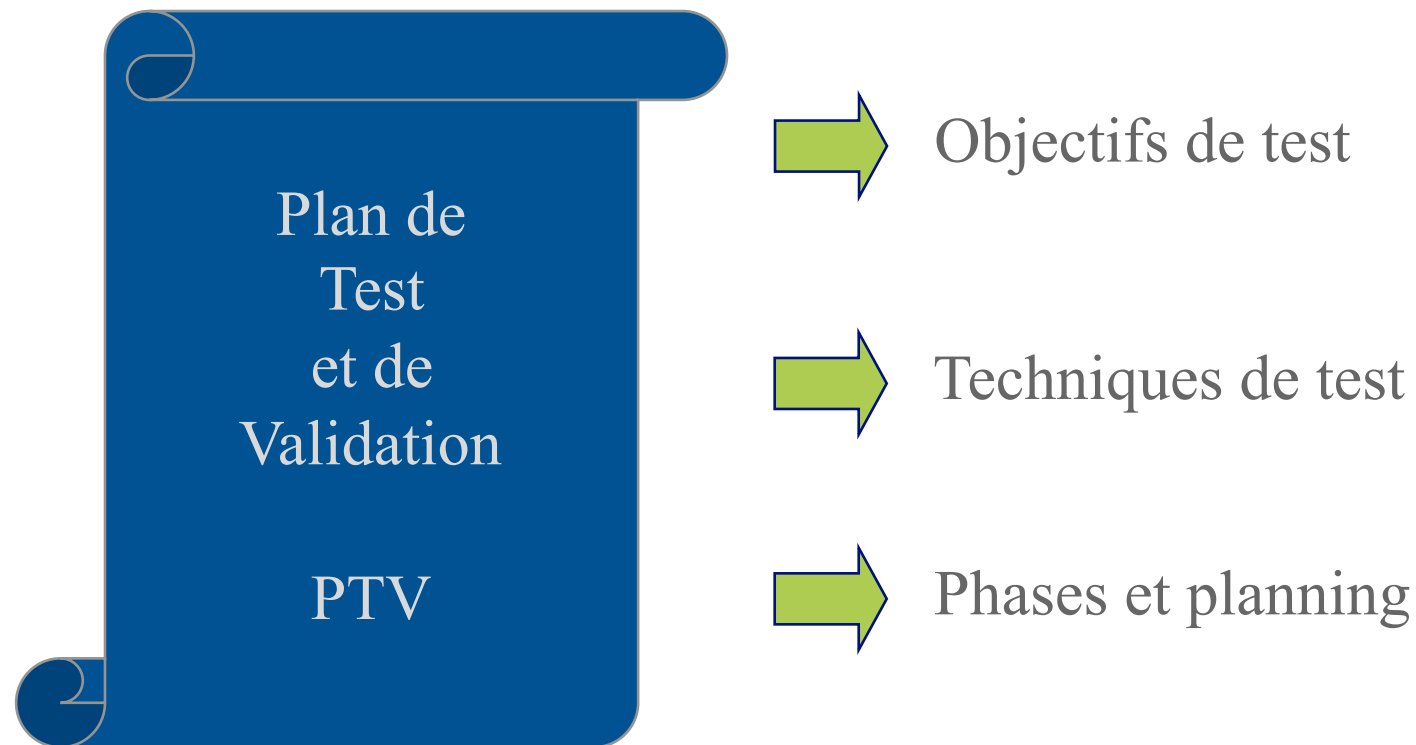


Le but est de les détecter le plus tôt possible (Stratégie)



Stratégie de test

En début de projet, définition d'un Plan de Test et de Validation (PTV)

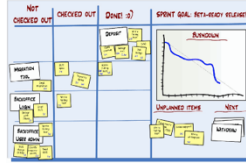


Acteurs du test



- Deux situations :
 - Je teste un programme que j'ai écrit
 - Je teste un programme que quelqu'un d'autre a écrit
- Trois questions :
 - Comment choisir la technique de test ?
=> **boite blanche** ou **boite noire** ?
 - Comment obtenir le résultat attendu ?
=> problème de **l'oracle** du test
 - Comment savoir quand arrêter la phase de test ?
=> problème de **l'arrêt**

Outils et Méthodes intervenants à différentes étapes

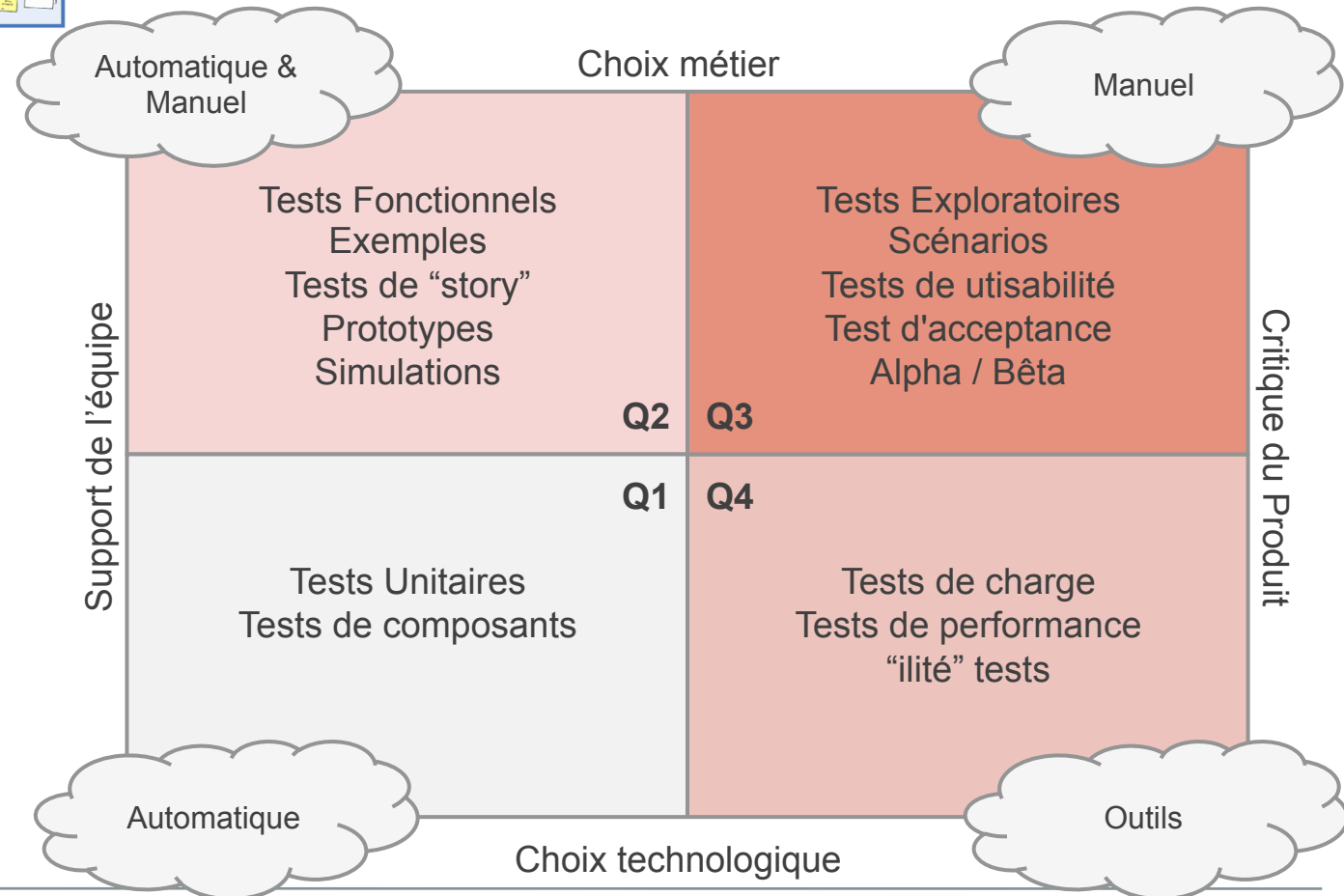


 mingle

 JIRA Agile

 ScrumDo

 hp Agile Manager



Outils et Méthodes intervenants à différentes étapes

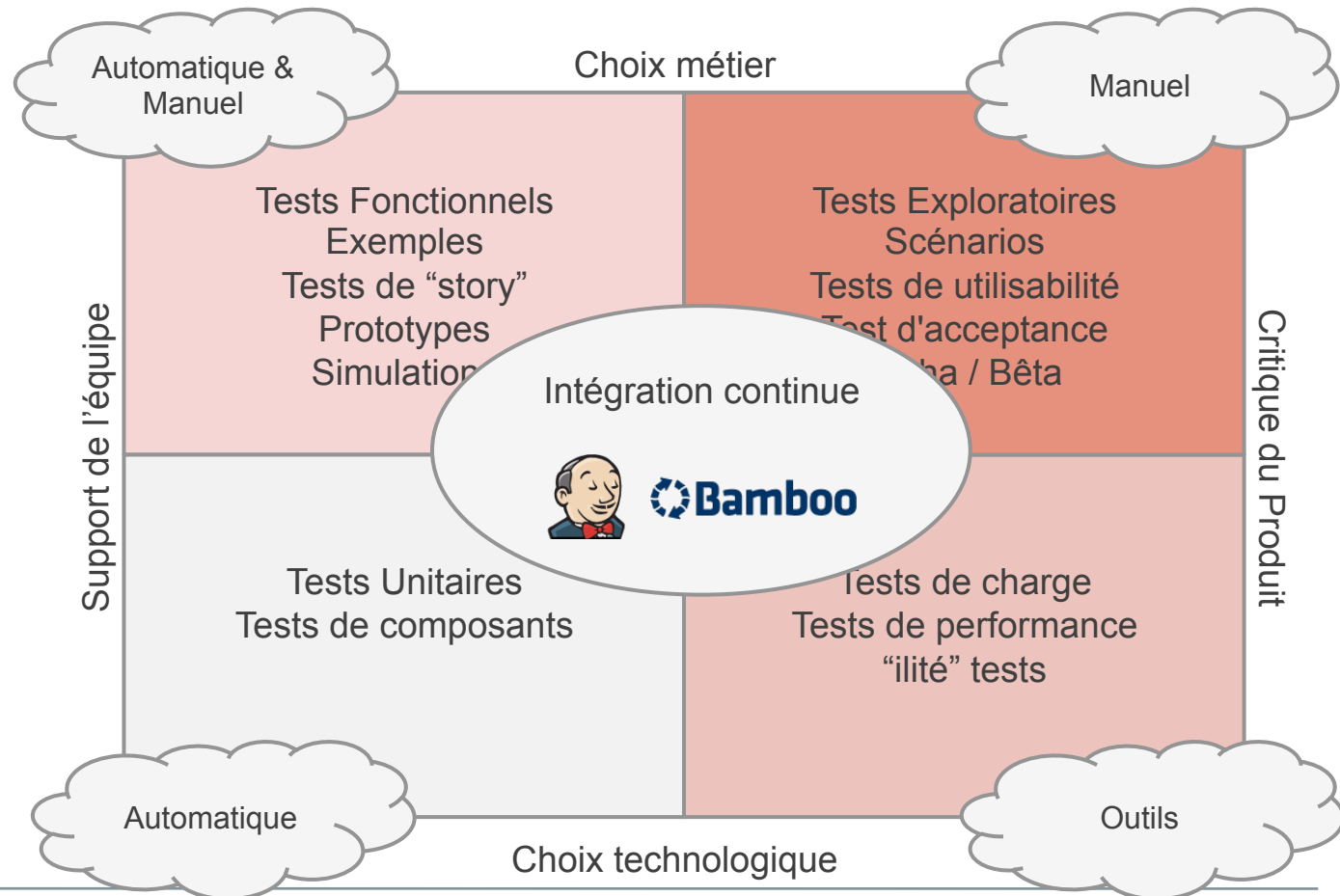
Exemple Mingle pour les projets Agile

The screenshot displays the Click n Dev PB 9 Storyboard - Mingle interface. The main area shows a Kanban board with columns: 'not set (2)', 'A faire (17)', 'En cours (56)', 'A tester (10)', 'A valider par PO (0)', and 'Terminé (0)'. The board is divided into two sections: 'Dev CnD' and 'Dev WSQC'. Cards are color-coded by complexity (1-13) and contain details like ID, title, and assignee. For example, in the 'A faire' column, there are cards for 'Consultation des indicateurs' (ID: #562) and 'Ordre les utilisateurs d'une' (ID: #1079). In the 'En cours' column, there are cards for 'Action des actions sur association' (ID: #1019) and 'Action des actions sur contextuelles' (ID: #1020). In the 'A tester' column, there is a card for 'Ajout d'une enveloppe OC à un ennet' (ID: #1075). In the 'A valider par PO' column, there is a card for '001R00002' (ID: #1116). In the 'Terminé' column, there is a card for 'Rapports les enveloppes par entité' (ID: #1073). The right sidebar shows filters and a color legend for complexity.

Team	not set (2)	A faire (17)	En cours (56)	A tester (10)	A valider par PO (0)	Terminé (0)	
Dev CnD	[OC] #790 Interface avec CCave	[OC] #562 Consultation des indicateurs	[GDA] #1019 Action des actions sur association	[OC] #1075 Ajout d'une enveloppe OC à un ennet			
	#1131 [Evo]Ordo Ergonomie - améliorations	[OC] #1079 Ordre les utilisateurs d'une	[GDA] #1020 Action des actions sur contextuelles	#1116 001R00002			
		#1090 [ANOS] Correction des spots	[ERG] #1096 relation page d'accueil V2				
			[ERG] #1111 Page accueil V2 - Notifications				
			[ERG] #1112 Page accueil V2 - Notifications				
			[ERG] #1113 Page accueil V2 - Notifications				
			[ERG] #1114 Page accueil V2 - Notifications				
			[ERG] #1115 Page accueil V2 - Notifications				
	Dev WSQC	[MSGC] #1070 Ajouter la possibilité de filtrer les	[MSGC] #1039 Etudier les caractéristiques de	[MSGC] #1032 Océan des index			[MSGC] #1073 Rapports les enveloppes par entité
		[MSGC] #1091 Modifier l'existence de translation		#1092 [MSGC] Création des associations			
[MSGC] #1144 Opération de traduction			[MSGC] #1094 Traduction				

Outils et Méthodes intervenants à différentes étapes

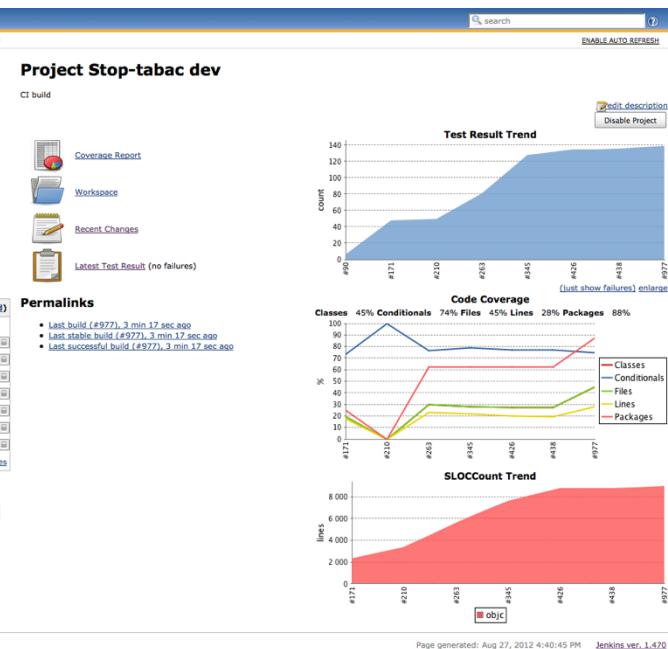
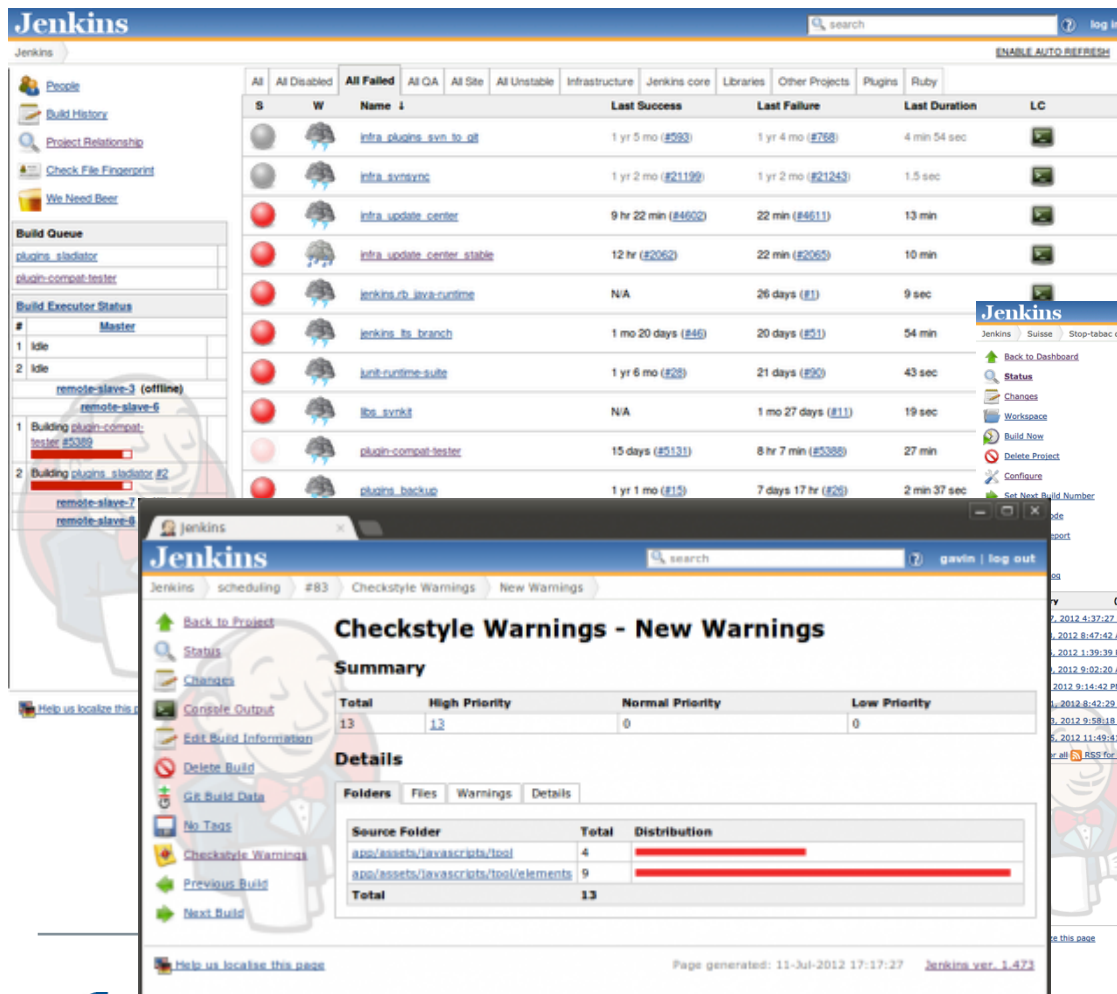
Build et Distribution: Utile pour tout le quadrant de test



Outils et Méthodes intervenants à différentes étapes

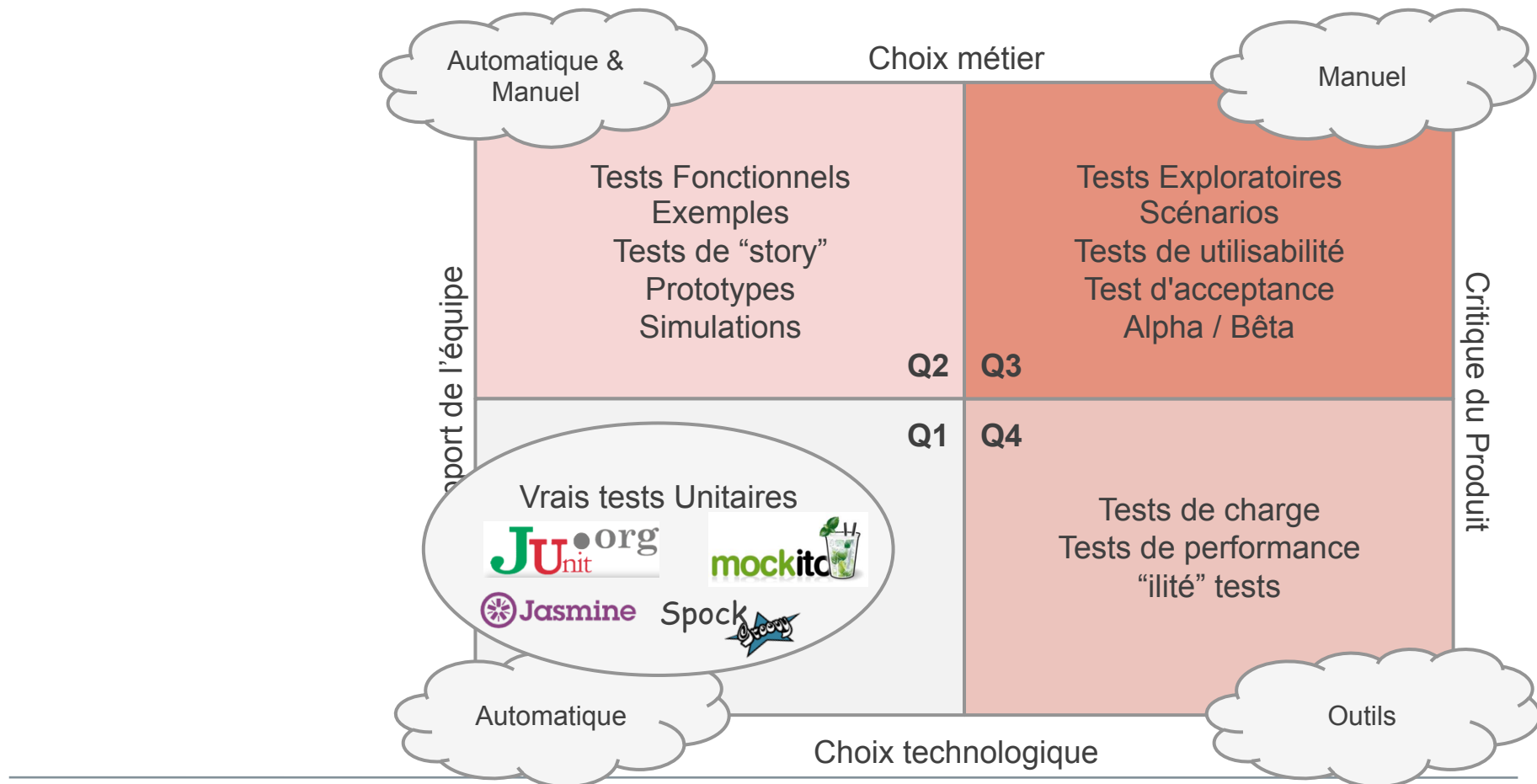
Intégration continue : Tableau de contrôle

Permet d'avoir en temps réel le statut du projet (tests unitaires, tests statiques, couverture...)



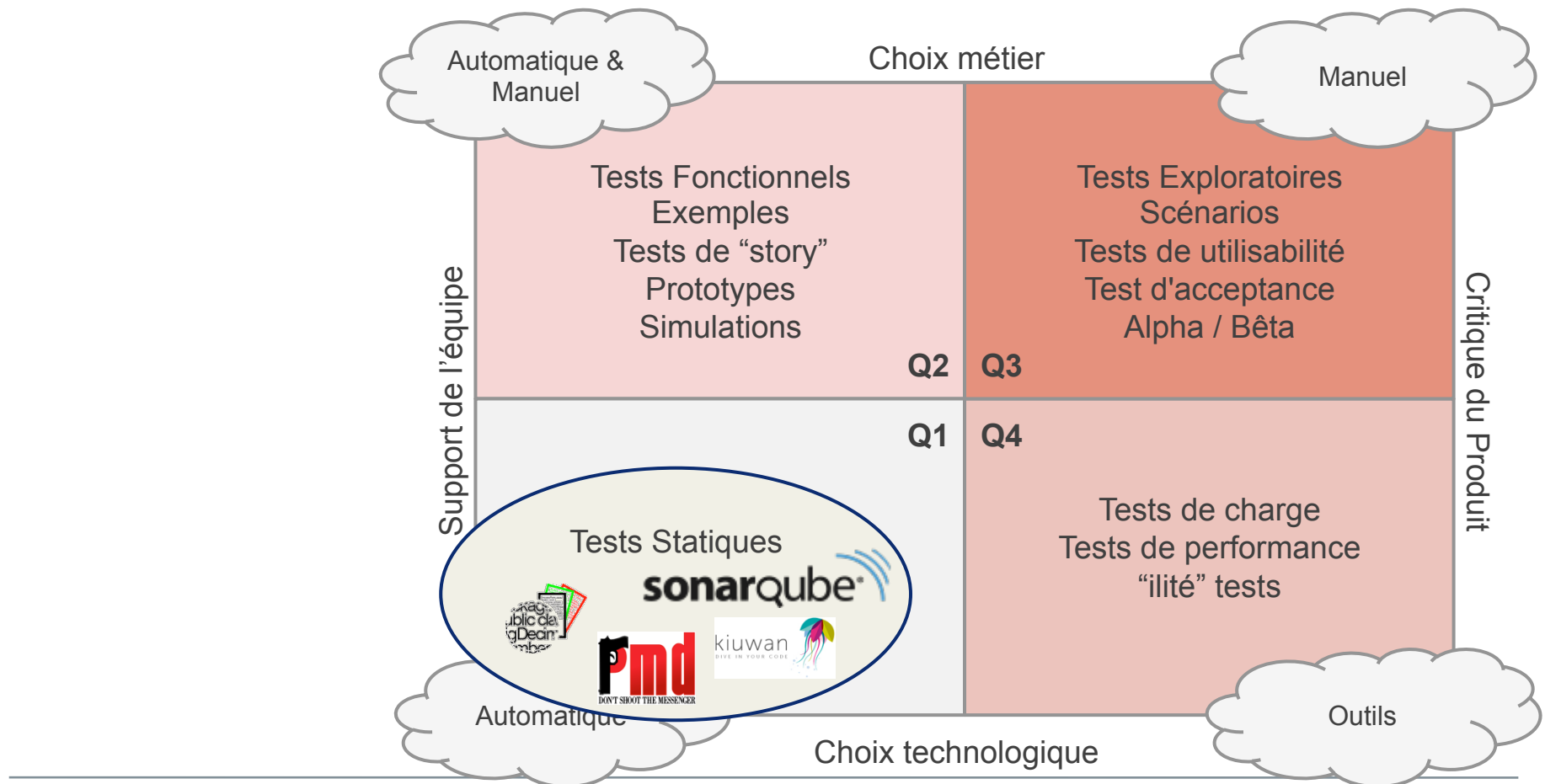
Outils et Méthodes intervenants à différentes étapes

Tests unitaires



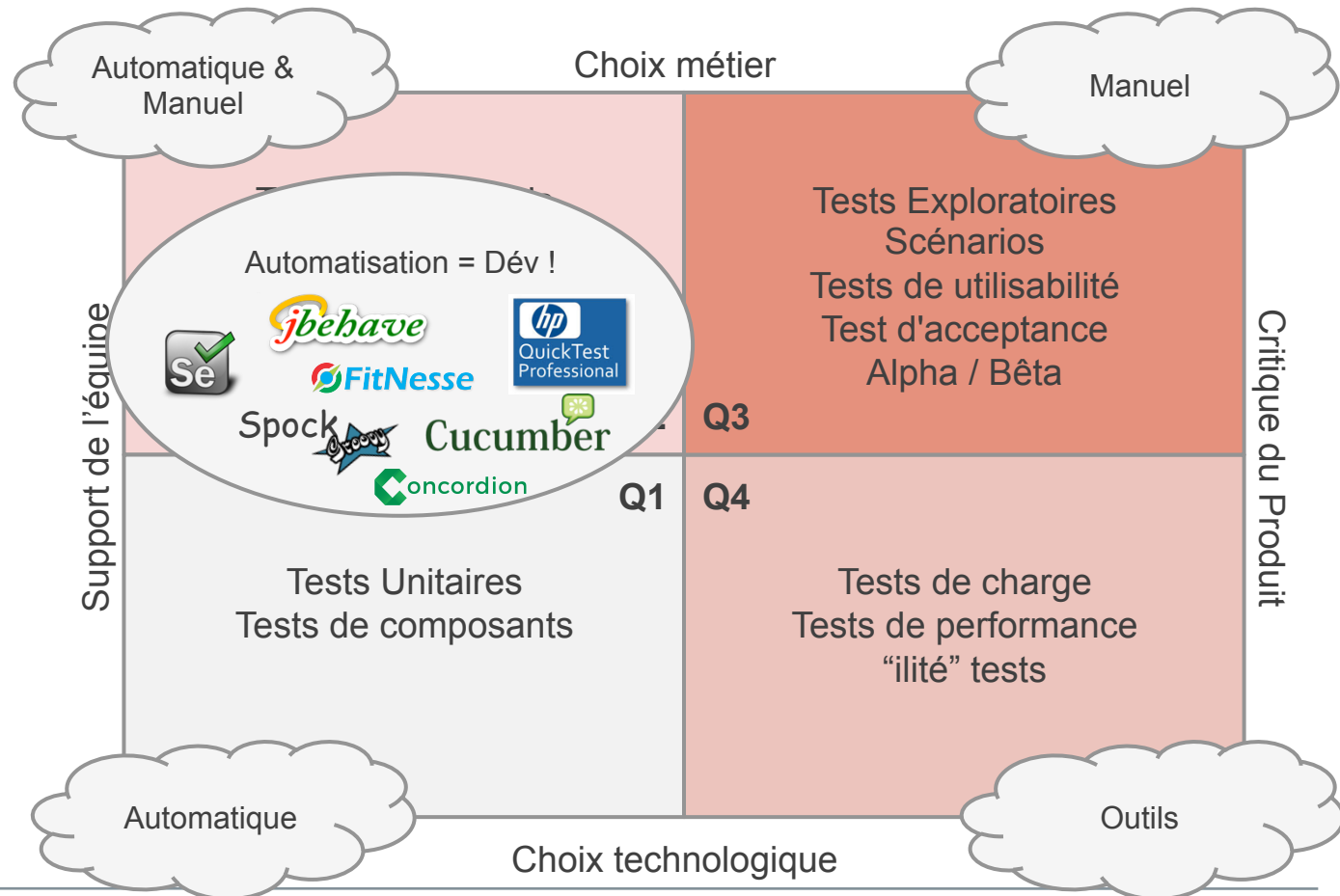
Outils et Méthodes intervenants à différentes étapes

Tests statiques !



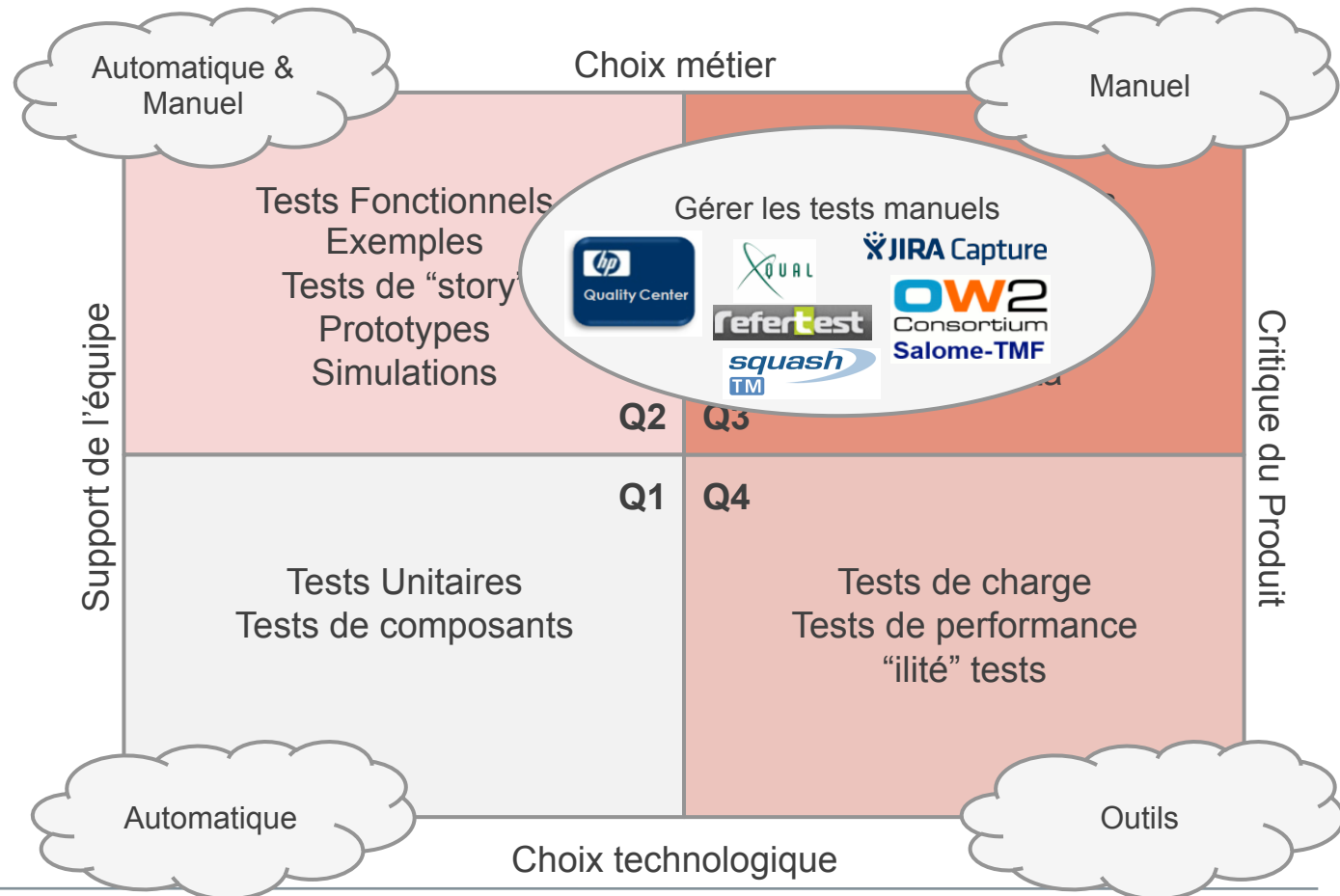
Outils et Méthodes intervenants à différentes étapes

Automatisation des tests



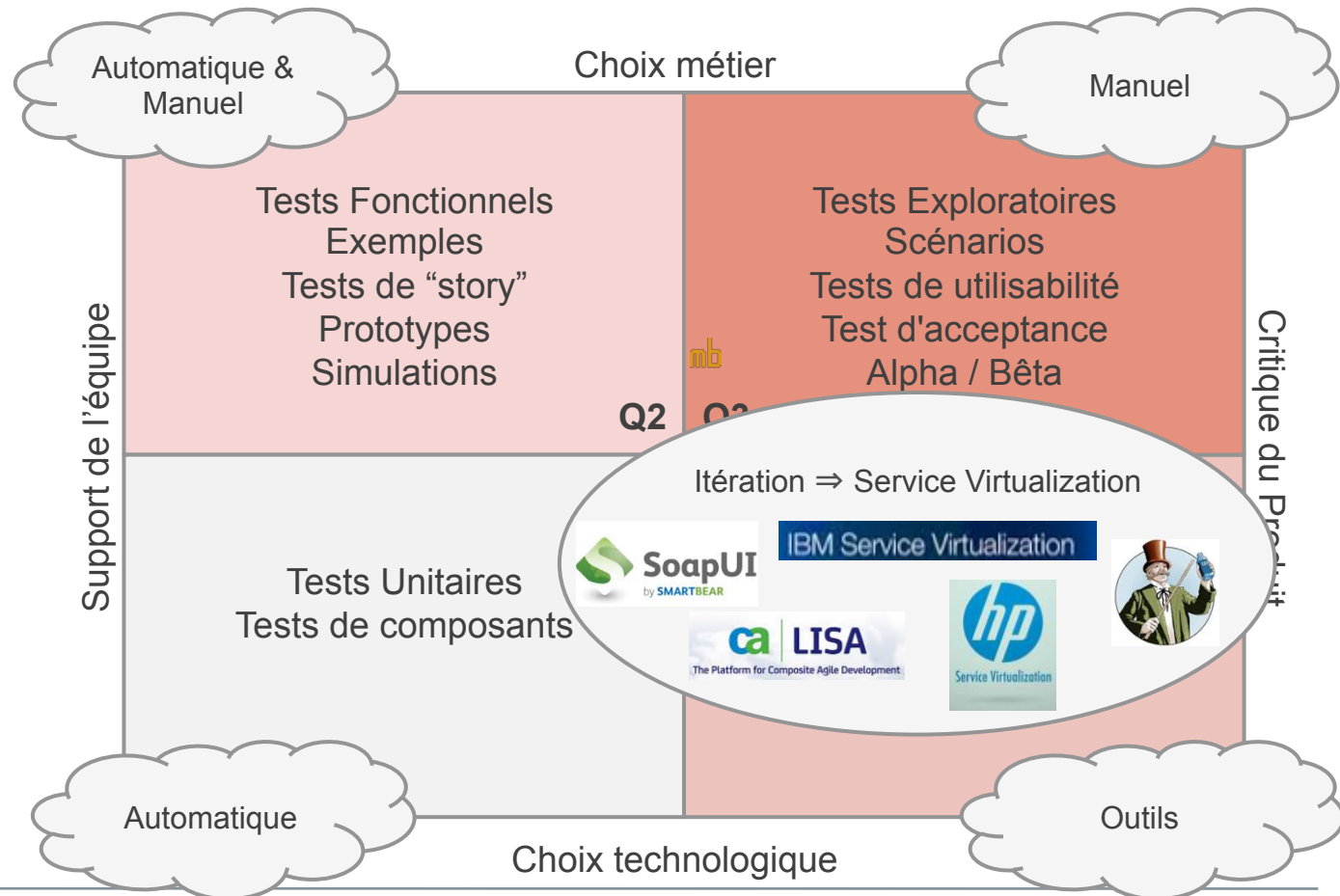
Outils et Méthodes intervenants à différentes étapes

Gestion des tests y compris exploratoires !



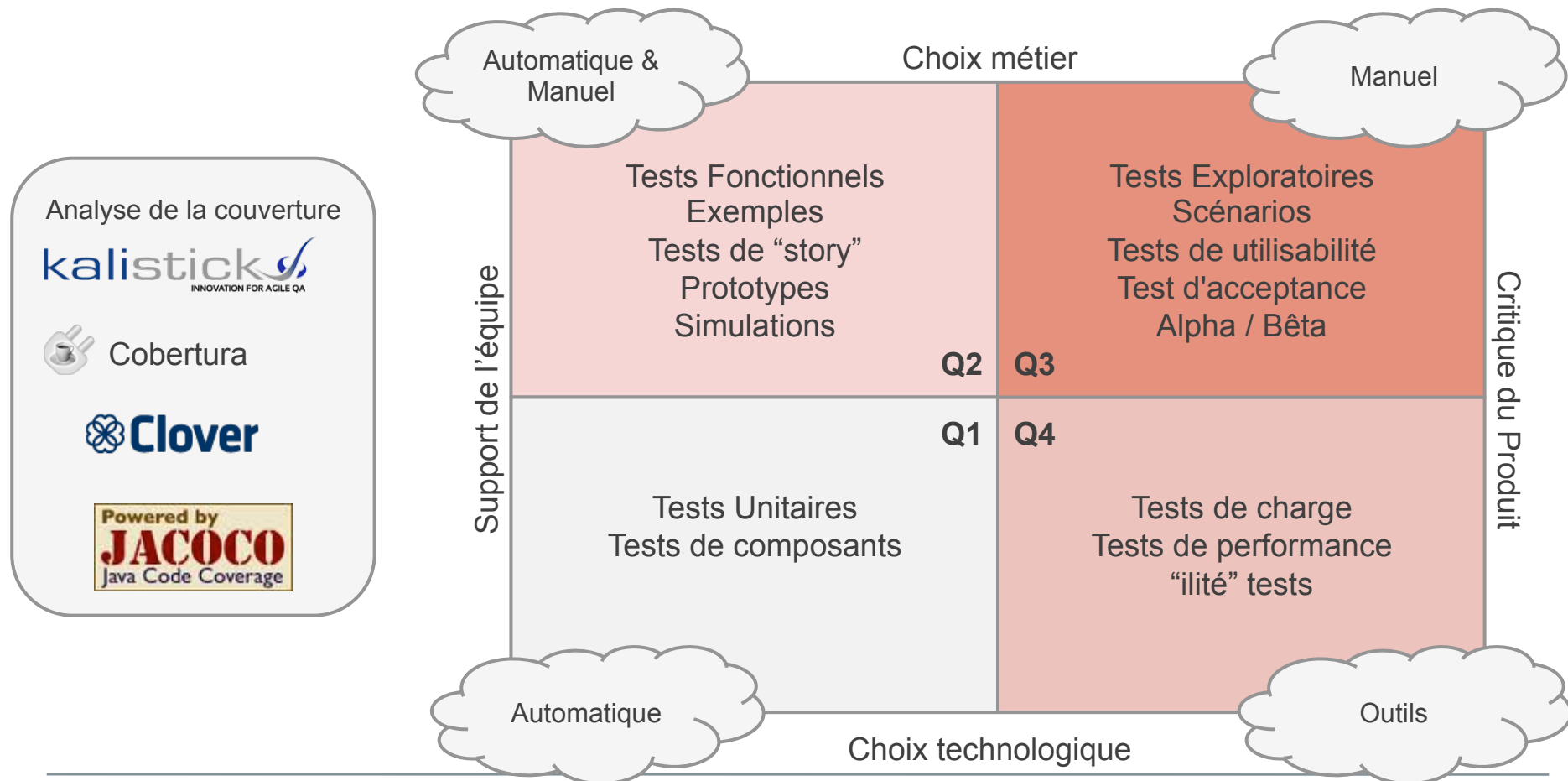
Outils et Méthodes intervenants à différentes étapes

Virtualisation et Cloud : un nouveau défi !



Outils et Méthodes intervenants à différentes étapes

Couverture : un métrique important



Outils et Méthodes intervenants à différentes étapes

BDD avec JBehave

1. Write story

Plain
text

Scenario: A trader is alerted of status

Given a stock and a threshold of 15.0

When stock is traded at 5.0

Then the alert status should be OFF

When stock is traded at 16.0

Then the alert status should be ON

2. Map steps to Java

POJO

```
public class TraderSteps {  
    private TradingService service; // Injected  
    private Stock stock; // Created  
  
    @Given("a stock and a threshold of $threshold")  
    public void aStock(double threshold) {  
        stock = service.newStock("STK", threshold);  
    }  
    @When("the stock is traded at price $price")  
    public void theStockIsTraded(double price) {  
        stock.tradeAt(price);  
    }  
    @Then("the alert status is $status")  
    public void theAlertStatusIs(String status) {  
        assertThat(stock.getStatus().name(), equalTo(status));  
    }  
}
```

Outils et Méthodes intervenants à différentes étapes

Tester avec FitNesse

- FitNesse utilise un wiki pour définir les cas de tests : on écrit des tables de décision. FitNesse est bien adapté à des testeurs fonctionnels
- Le développeur doit écrire le “fixture” code pour appeler les fonctionnalités concernées

eg.triviaGameExample.fitnessFixtures.AddRemovePlayerFixture

playerName	addPlayer?	countPlayers?
Al	true	1
Bertha	true	2

```
import fit.ColumnFixture;

public class AddRemovePlayerFixture {
    private String playerName;
    private Game theGame;

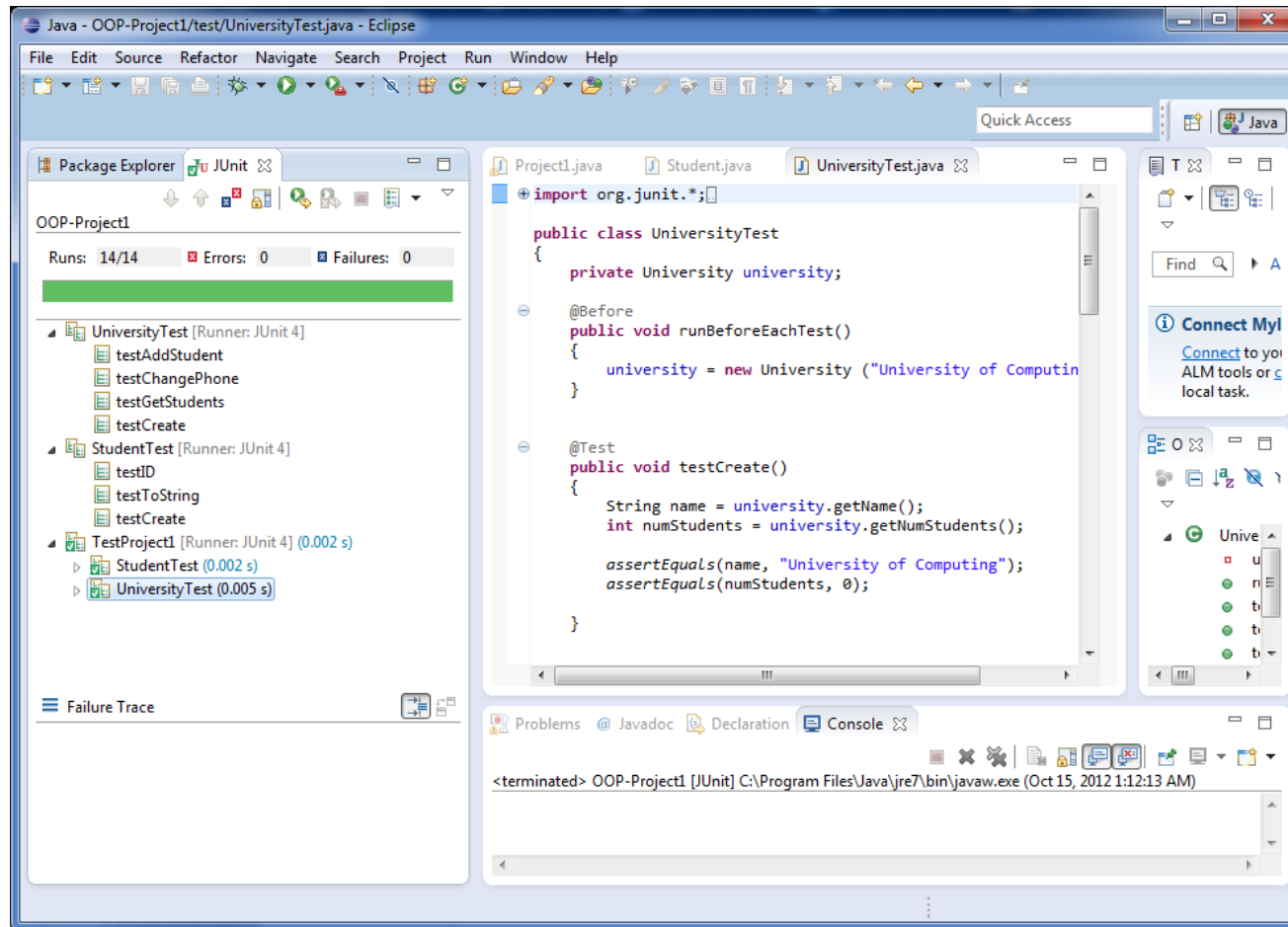
    public void setPlayerName(String playerName) {
        this.playerName = playerName;
    }

    public boolean addPlayer() {
        theGame = StaticGame.theGame;
        Player thePlayer = theGame.addPlayer(playerName);
        return theGame.playerIsPlaying(thePlayer);
    }

    public int countPlayers() {
        return theGame.getNumberOfPlayers();
    }
}
```

Outils et Méthodes intervenants à différentes étapes

Exemple: tests unitaires avec Junit depuis Eclipse



Outils et Méthodes intervenants à différentes étapes

Exemple: qualité du code mesurée avec Sonar

